You could solve this problem by extracting greet from MyClass into its own module, like the Greetings module in the previous section. If you do that, you'll be able to insert an intermediary module like EnthusiasticGreetings in the chain and use the override-and-call-super technique, just as we did back then. However, you might be unable to do that—for example, because MyClass is part of a library such as Rails, and you're extending that library rather than working directly on its source code. This limitation is the main reason why many Rubyists still use alias_method_chain when they extend Rails.

However, Ruby 2.0 came with an elegant solution for this problem in the form of Module#prepend:

```
module EnthusiasticGreetings
  def greet
    "Hey, #{super}!"
  end
end

class MyClass
  prepend EnthusiasticGreetings
end

MyClass.ancestors[0..2]  # => [EnthusiasticGreetings, MyClass, Object]
MyClass.new.greet        # => "Hey, hello!"
```

This is a *Prepended Wrapper (136)*, a modern alternative to *Around Aliases (134)*. Because we used prepend, the EnthusiasticGreetings#greet got lower than MyClass#greet in MyClass's chain of ancestors, so we went back to the usual trick of overriding greet and calling super.

As I write, Rails is not using Module#prepend yet, because it's still aiming to be compatible with Ruby 1.9. When Rails eventually drops this constraint, I expect that prepend will make its appearance in Rails and its extensions. At that point, there will be no urgent reason to call alias_method_chain anymore.

A Lesson Learned

Throughout this book I showed you how convenient, elegant, and cool metaprogramming can be. The story of alias_method_chain, however, is a cautionary tale: metaprogramming code can sometimes get complicated, and it can even cause you to overlook more traditional, simpler techniques. In particular, sometimes you can avoid metaprogramming and use plain, old-fashioned object-oriented programming instead.

The lesson I personally learned from this story is: *resist the temptation to be too clever in your code*. Ask yourself whether there is a simpler way to reach